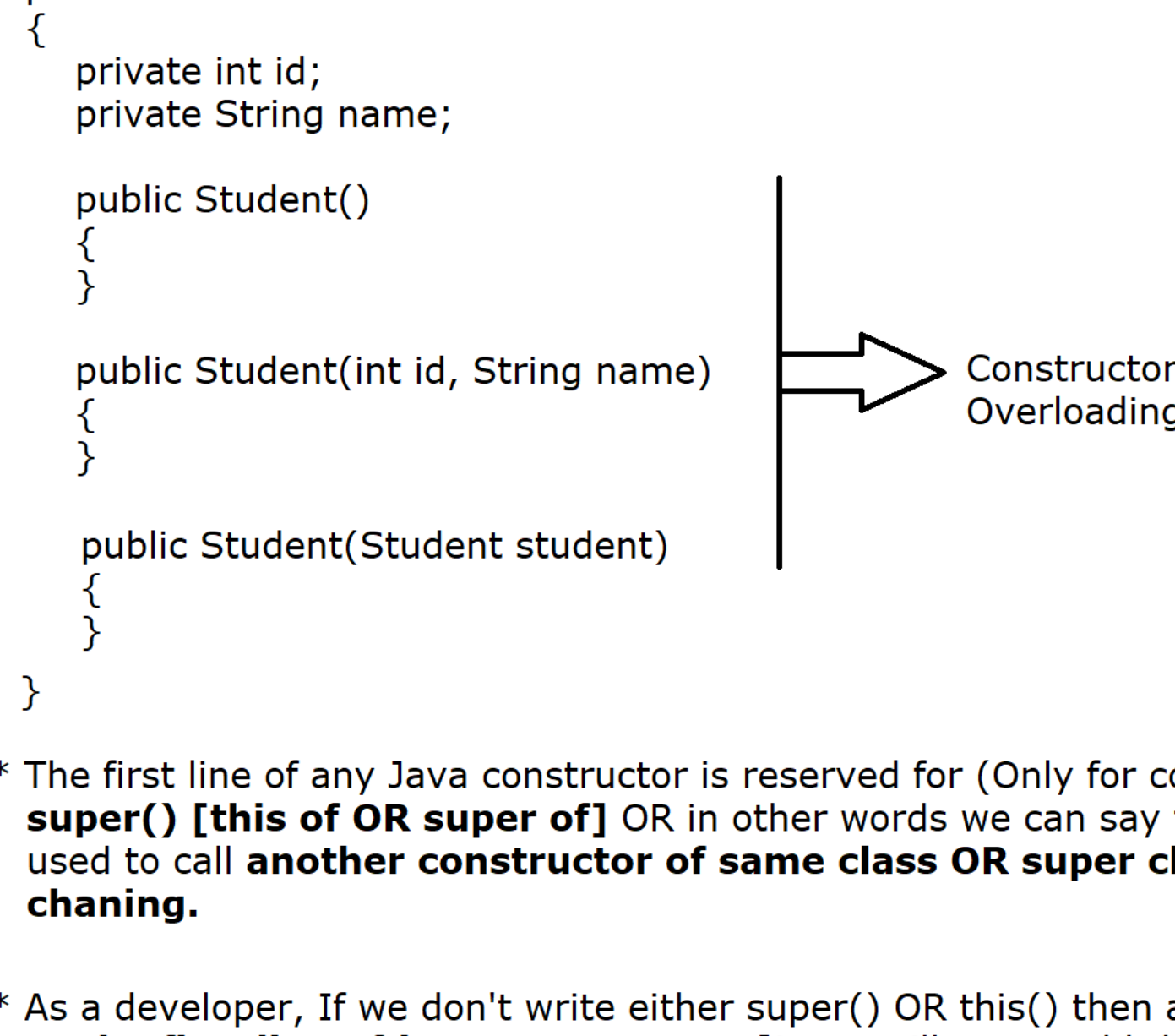


Constructor Overloading :

In Java, It is possible to write multiple constructors in the same class (only in the same class, Method overloading is possible in super & sub class) where constructors name must be same but the parameter must be different by using

- a) Number of Parameters
- b) Type of Parameters
- c) Order of Parameters

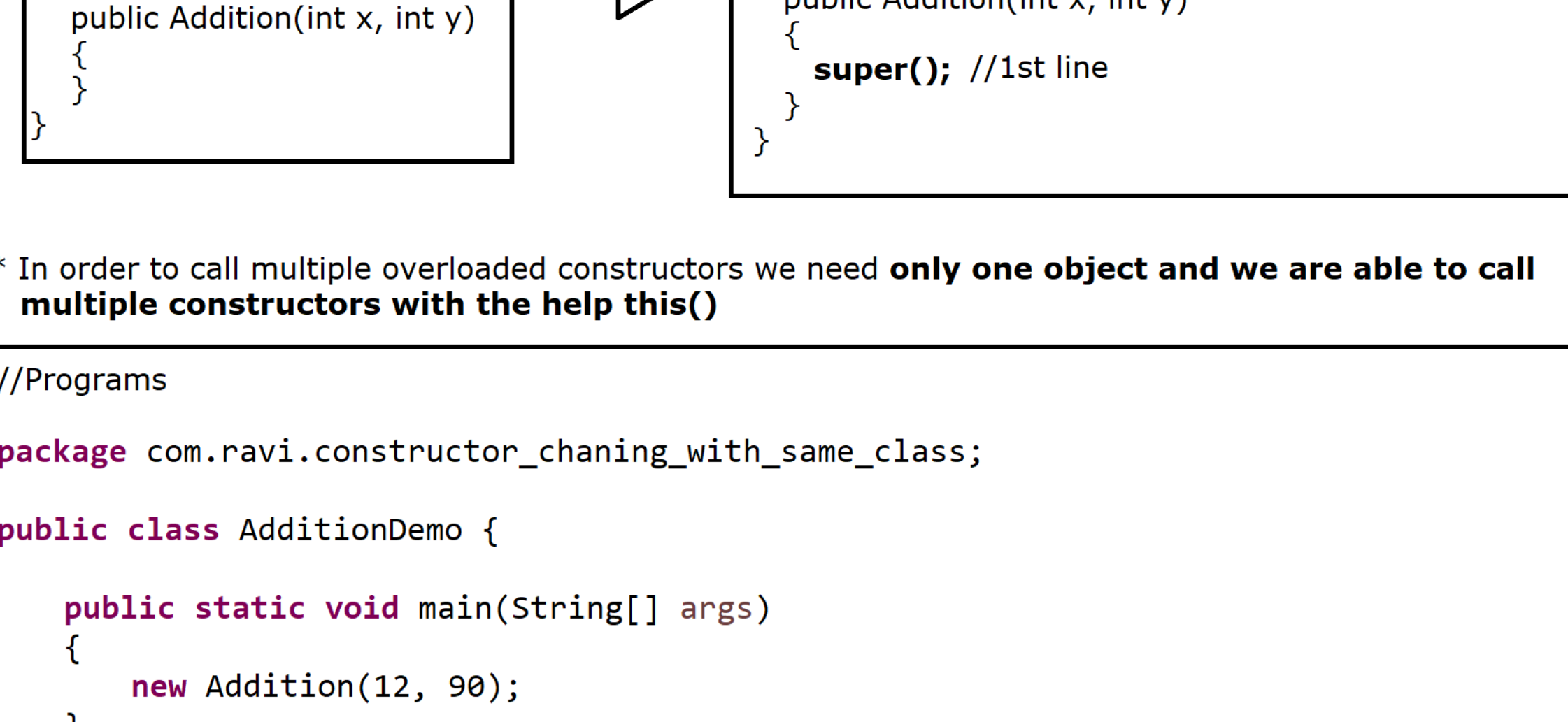
Example :



* The first line of any Java constructor is reserved for (Only for constructor not for method) either **this()** OR **super()** [this of OR super of] OR in other words we can say the First line of any java constructor is used to call **another constructor of same class OR super class which is known as Constructor chaning**.

* As a developer, If we don't write either super() OR this() then automatically java compiler will add **super()** to the first line of java constructor. [javac will never add this(), It is developer responsibility]

Example :



* In order to call multiple overloaded constructors we need **only one object and we are able to call multiple constructors with the help this()**

//Programs

```
package com.ravi.constructor_chaning_with_same_class;

public class AdditionDemo {

    public static void main(String[] args)
    {
        new Addition(12, 90);
    }
}

class Addition
{
    public Addition(int x, int y)
    {
        this(2.4, 3.9);
        IO.println("Sum of two integer is :"+(x+y));
    }

    public Addition(double x, double y)
    {
        this("Data", "base");
        IO.println("Sum of two double is :"+(x+y));
    }

    public Addition(String x, String y)
    {
        IO.println("Concatenation of String is :"+(x+y));
    }
}
```

Another Program :

```
public class ConstructorChaningDemo2
{
    public static void main(String[] args)
    {
        new Employee(101);
    }
}

class Employee
{
    public Employee(int id)
    {
        this(101, "Scott");
    }

    public Employee(int id, String name)
    {
        this(101, "Scott", 45000);
    }

    public Employee(int id, String name, double salary)
    {
        this(101, "Scott", 45000, "Hyderabad");
    }

    public Employee(int id, String name, double salary, String address)
    {
        super();
        IO.println("Id is :"+id);
        IO.println("Name is :"+name);
        IO.println("Salary is :"+salary);
        IO.println("Address is :"+address);
    }
}
```

Modifiers on Constructor :

* We can apply any kind of access modifier on the constructor like private, default (package - private), protected & public

* We cannot apply the following modifiers

- a) static : Constructor plays a major role in object creation
- b) final : Constructors are available in the same class only (not in sub class) so no point to declare final
- c) abstract : abstract method cannot have method body but constructor must have body.

IQ :

When we should declare a constructor with private access modifier ?

We can declare a constructor with private access modifier in the following two cases :

- 1) If we want to return the Object to the client by using static factory method [Singleton design pattern] then we can declare a constructor as private.
- 2) If we want to declare **only static Fields and static methods** inside a class then we should declare constructor with private access modifier [Object is not required]

Example : java.util.Arrays, java.lang.Math and java.util.Collections class has private constructor because It contains only static field and static methods.[Utility class]

Non static block OR Instance Initializer :

* It is a special block in java which we can write **inside a class**, It will be automatically executed when we create an object for the class.

Example :

```
{
    //Non static block
}

//Program
package com.ravi.non_static_block;

class Test
{
    {
        IO.println("Non static block");
    }
}

public class NSBDemo1
{
    public static void main(String[] args)
    {
        IO.println("Main");
        new Test();
        new Test();
    }
}
```